



Theoretical Foundations of Artificial Intelligence

Contents

Unit VI A – Applications (NLP)

1. Natural Language Processing (NLP)
2. Steps in NLP
3. Syntactic Processing and Augmented Transition Nets
4. Semantic Analysis and grammars
5. Discourse and pragmatic processing

Unit VI B – Applications (Planning Systems)

1. What is Planning System in AI
2. Components of a Planning System
3. Goal Stack Planning
4. Non-linear planning using constraint posting
5. Hierarchical Planning
6. Reactive Systems

Unit VI C – Applications (Expert Systems)

1. Expert Systems
2. Applications of Expert Systems
3. Components of an Expert System
4. Expert System Development
5. Program: Creating an Expert System using Python – Medical Diagnosis
6. Program: Creating an Expert System using Prolog – Medical Diagnosis
7. Types of Expert Systems
8. Rule-Based Expert System
9. Frame-Based Expert System
10. Neural Network Based Expert System
11. Blackboard Architecture
12. Case studies of successful expert Systems
13. Knowledge acquisition and Validation techniques

Section-A (NLP)

1. Introduction to Natural Language Processing (NLP)

Def: Making computers understand and process human language (text or speech).

Example: Google Translate.

Think of NLP as teaching a robot to understand English (or any language) just like teaching a child — step by step.

Example1:

Input: **"Raju ate a Mango."**

NLP tries to understand:

- Who? → Ravi
- What action? → ate
- What object? → a Mango

Example2:

When you ask a voice assistant, **"What is the weather like tomorrow?"** the assistant uses NLP to understand that:

- **"What"** → A request for information.
- **"weather"** → The topic of the request.
- **"tomorrow"** → The time frame.

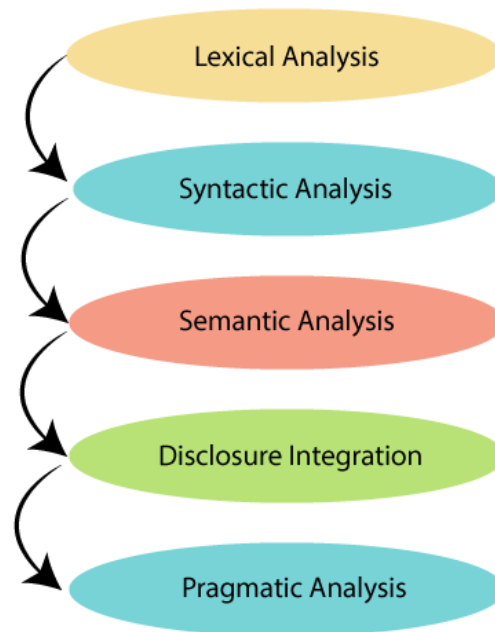
It then converts this into a structured query like GET (weather, date=tomorrow).

Uses of NLP

- AI Chatbots – Web Hosting – Looking for different options – Use AI Enabled Chatbot
- Speech assistants (Siri, Google Assistant)
- Translation systems (like Google Translate)
- Email spam detection
- Sentiment Analysis (Amazon Product Reviews, Hotel Reviews)
- Text summarizers (Law, Medicine etc)

2. Natural Language Processing Steps

NLP is not a single step; it is a pipeline of processes. Think of it like a factory assembly line where a raw material (your sentence) is progressively refined into a finished product (meaning).



Let us understand one by one:

Lexical Analysis and Morphological Analysis

Lexical Analysis

Split the sentence into words.

Example:

“Ravi ate an apple.” → [Ravi], [ate], [an], [apple]

Morphological Analysis

Find the root (stem) of each word.

Example:

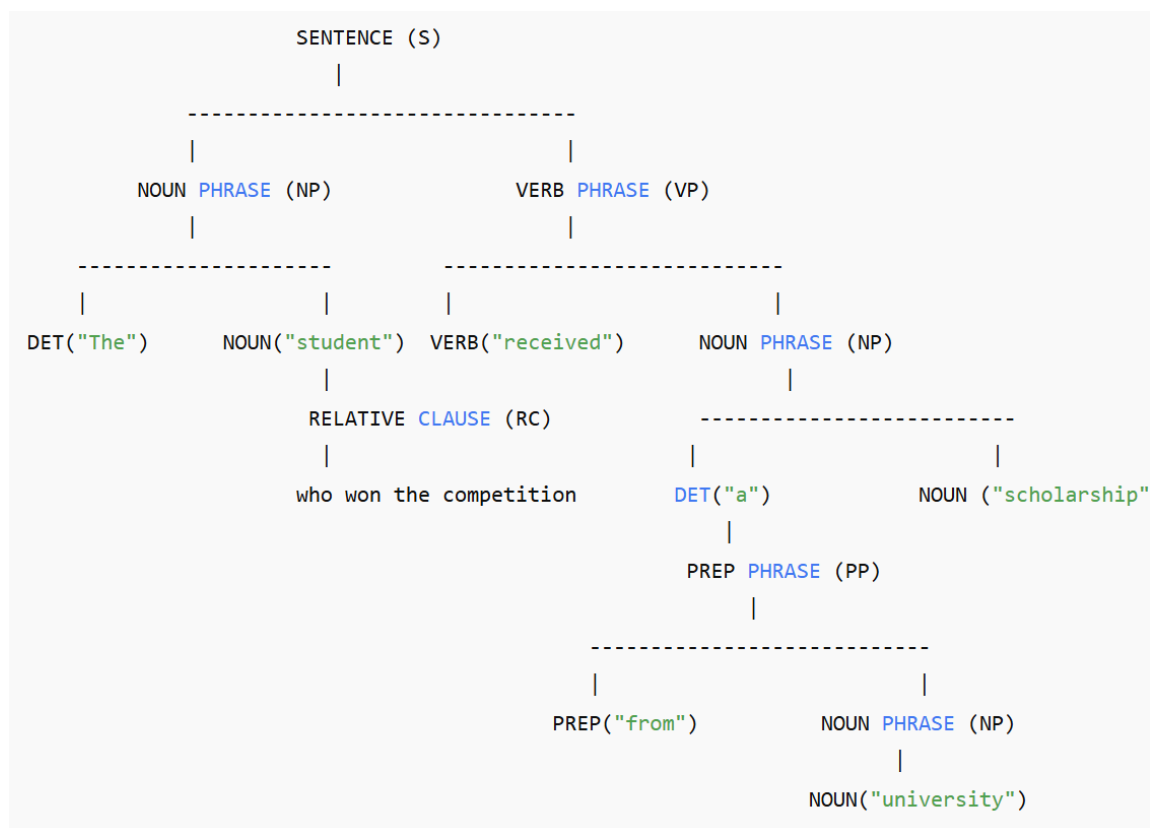
- “eating”, “eats”, “ate” → **eat**

Syntactic Processing (Parsing)

Determine grammar structure (like subject–verb–object).

Example Sentence:

“The student who won the competition received a scholarship from the university.”



What Does Syntactic Processing Give the AI?

It provides:

- **Subject** → “The student who won the competition”
- **Main Verb** → “received”
- **Object** → “a scholarship”
- **Source** → “from the university”
- **Embedded clause** → who won the competition → tells **WHICH** student

The system now understands the structure without understanding meaning.

Semantic Analysis

Understand the **actual meaning**.

“Apple” → fruit, not Apple Inc.

Discourse Analysis

Connect meaning across sentences.

Example:

“Ravi ate an apple. **He** liked it.”

“He” = Ravi

“it” = apple

Pragmatic Analysis

Understand **real intention**.

Example:

“Can you pass the salt?”

→ It’s not a question about ability

→ It is a request

Semantic Vs Pragmatic Analysis

Example sentence: “It’s already 8:30 and the meeting was supposed to start at 8.
Are you planning to join us today?”

Semantic Analysis (Literal Meaning)

The system extracts:

- Time is 8:30
- Meeting should have started at 8
- The speaker is asking the listener if they plan to join

Semantic Output:

```
TIME(NOW = 8:30)
MEETING_START = 8:00
QUESTION: "Will you join the meeting?"
```

No emotion

No sarcasm

No frustration

No pressure

Just facts and grammar.

Pragmatic Analysis (Intended Meaning)

Pragmatics interprets the speaker’s **real intention**.

Here, the speaker is:

- indirectly **complaining**,
- suggesting the listener is **late**,
- prompting the listener to **join quickly**,
- possibly frustrated or sarcastic.

The question is NOT a real question;
It is a **polite but indirect reminder**.

Pragmatic Output:

Speaker's intention:

- You are late
- Please join the meeting immediately
- I'm slightly frustrated
- This is not a question; it's a reminder/complaint

3. Syntactic Processing and Augmented Transition Nets (ATNs)

What is Syntactic Processing?

It checks if a sentence follows **grammar rules**. So, It is the set of rules for building a valid sentence.

Example:

✓ "The dog chased the cat."

✗ "Dog the cat chased."

Augmented Transition Nets

It is a **graph-based method** for parsing sentences.

Sentence: "The student who topped the exam received a medal."

We use a small ATN with these subnets (networks):

- **S** (Sentence network) — top level
- **NP** (Noun Phrase) — parses noun phrases (subject / object)
- **RC** (Relative Clause) — parses clauses like "who topped the exam" (implemented as part of NP or as a callable subnet)
- **VP** (Verb Phrase) — parses the predicate (verb + object / complements)

Each subnet is a little state machine. Transitions can:

- consume a word category (Det, Noun, Verb, Pronoun, Prep, Adj, etc.)
- **CALL** another subnet (push return point)
- **RETURN** to caller on success
- perform **tests** (e.g., number agreement)
- store values into **registers** (like subject, verb, tense)

Parse Tree

```

S
├ NP (The student [who topped the exam])
│   ├── Det: The
│   ├── Noun: student
│   └── RC: who topped the exam
│       ├── RelPronoun: who (binds to student)
│       ├── Verb: topped
│       └── NP: the exam
│           ├── Det: the
│           └── Noun: exam
└ VP (received a medal)
    ├── Verb: received
    └── NP (a medal)
        ├── Det: a
        └── Noun: medal

```

High-level parsing plan

1. S calls NP to parse the subject.
2. NP recognizes "The student who topped the exam" as one NP (with an embedded relative clause).
3. Return to S. S calls VP to parse the predicate "received a medal".
4. VP parses verb and object NP.
5. S (optionally) performs agreement checks (subject-verb number(singular/plural)) and then accepts sentence.

Let's see how agreement check *catches* an error:

✗ "The student who topped the exam **receive** a medal."

- NP head = **student** (singular)
- Verb = **receive** (plural form)

Check fails → Parser rejects.

4. Semantic Analysis and Semantic Grammars

When a sentence is parsed syntactically, we get **only the structure**:

- What is the subject?
- What is the verb?
- What is the object?
- What are the phrases?

But **syntactic parsing alone cannot tell us the *meaning* of the sentence.**

That is where **Semantic Analysis** and **Semantic Grammars** come in.

Example: "The Chicken is ready to eat."

Two Different Meanings (Semantics)

Meaning 1 — The Chicken (animal) will eat something.

- Chicken = agent (doer)
- “ready to eat” = ready *to perform* the action of eating

Meaning 2 — The Chicken (food) is prepared for you to eat.

- Chicken = food
- “ready to eat” = prepared for *someone else* to eat

What Semantics Uses to Decide Meaning

Semantic rules may look at:

✓ World knowledge

- Raw chicken cannot eat.
- Cooked chicken is something humans eat.

✓ Lexical meaning

“ready to eat” can modify food (adjectival phrase).

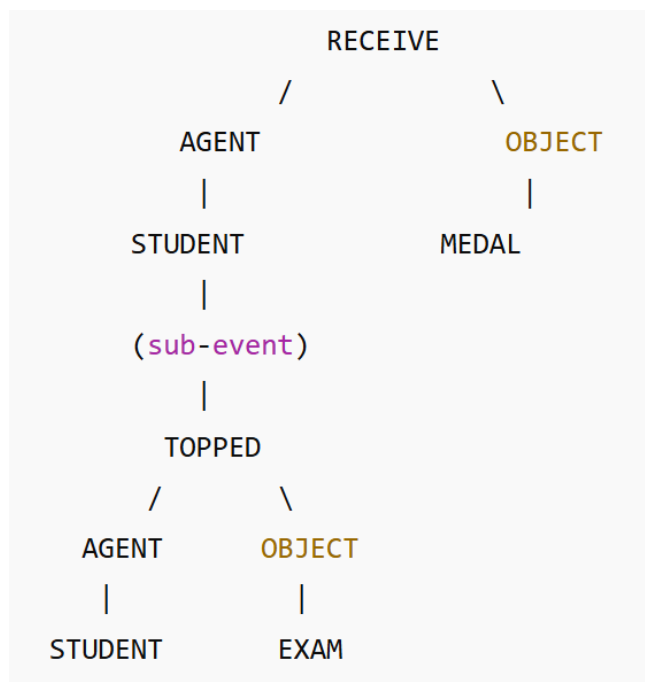
✓ Commonsense inference

We rarely talk about chickens waiting to eat.

Syntax cannot use any of these.

Example2: Sentence: “Visiting relatives can be annoying.”

Meaning Representation Graph



5. Discourse and Pragmatic Processing

Discourse Processing

Handling meaning across multiple sentences.

Example

“Ravi went to the shop. He bought milk.”

“He” refers to Ravi.

Pragmatic Processing

Understanding **context, intention, politeness, culture**, etc.

Example

Person A: “It’s cold here.”

Meaning: Indirectly asking to close the window.

What is Planning System in AI

Planning = Finding a sequence of actions to go **FROM state A TO state B**.

Example:

Goal: Make tea

Plan:

1. Boil water
2. Add tea leaves
3. Add milk
4. Serve

Components of a Planning System

A planning system usually has:

1. **States** – current situation
2. **Goals** – what you want
3. **Operators (Actions)** – what you can do (like Boil water, pour tea in the context of making tea, apply break, turn left/right in the context of Self Driving Car)
4. **Planner** – chooses actions
5. **Executor** – performs actions. After the planner produces the action list, the **executor executes them** in the real (or simulated) world.
6. **Monitor** – checks if actions succeeded. Eg: Sensor checks if the water temperature > 90°C
7. **Knowledge Base** – stores facts (like: “Water boils at 100°C”, “Robot cannot lift more than 2kg”)

Goal Stack Planning (Linear Planning)

Goal Stack Planning is a type of planning where the agent solves goals one at a time, in a stack (LIFO) order.

When a subgoal appears, it is pushed onto the stack. When it is solved, it is popped.

The planning moves **step-by-step in a straight line** (linear order).

Example: Classroom Assignment Submission

Final Goal:

Submit assignment on the college portal

Subgoals:

1. Finish writing the assignment
2. Convert to PDF
3. Upload to portal

Subgoal 1 (Finish writing assignment) has further subgoals:

- Research topic
- Prepare outline

- Write content

Planner's Goal Stack (Top → Bottom):

Goal Stack Plan (Sequential)

Final goal: Submit Assignment

So, the planner expands this into required subgoals:

Dependency Chain:

Submit-Assignment

↑

Upload-to-Portal

↑

Convert-to-PDF

↑

Finish-Assignment

↑

Write-Content

↑

Prepare-Outline

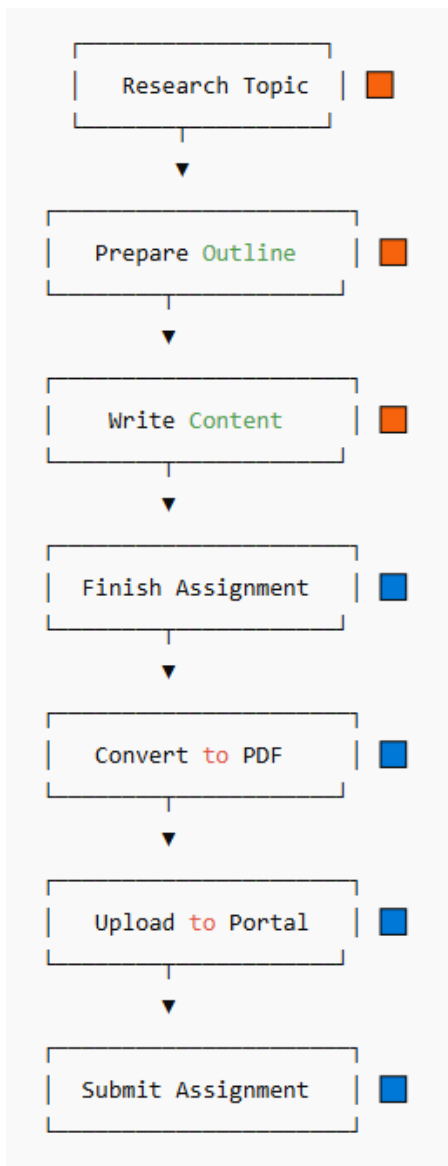
↑

Research-Topic

Now pop completed tasks in correct order:

Pop: Research-Topic → done
Pop: Prepare-Outline → done
Pop: Write-Content → done
Pop: Finish-Assignment → done
Pop: Convert-to-PDF → done
Pop: Upload-to-Portal → done
Pop: Submit-Assignment → done

Stack becomes empty → **Plan success ✓**



The agent solves things **one goal at a time**, in the sequence forced by the stack.

THE PLANNER WILL DO IT LIKE THIS:

Research → Outline → Content → Convert → Upload → Submit
(A single straight path.)

Characteristics of Goal Stack Planning

- Goals are solved **one at a time**
- Stack = last-in, first-out
- Order is **strict**
- Suitable for simple, linear problems
- Cannot handle parallel or partially independent actions

Non-linear planning using constraint posting

Non-linear planning (also called **partial-order planning**) allows multiple subgoals to be achieved in *any order*, as long as constraints are satisfied.

The planner does **not** force a strict sequence. Instead, it creates a **flexible plan** with ordering constraints.

Example: Classroom Exam Preparation

Goal: **Prepare for exam**

Subgoals:

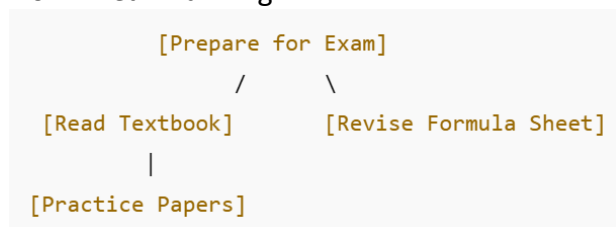
1. Read textbook
2. Practice previous papers
3. Revise formula sheet

All three can be done in any order!

But **constraints** may be:

- Must read textbook before practicing papers
(since you need concepts first)

Non-Linear Planning:



You do NOT get a strict fixed order like: Read → Revise Formula → Practice (NO!)

Hierarchical Planning (HTN)

Hierarchical = Breaking large goals into **sub-goals**, like a tree.

Example

Goal: “Plan a birthday party”

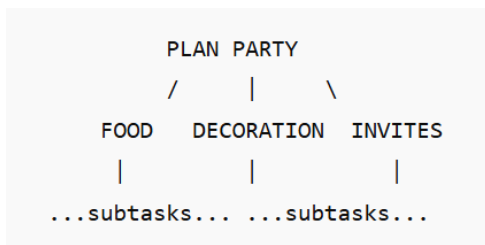
Break into:

1. Arrange food
2. Arrange decoration
3. Invite guests

Each has sub-tasks:

Invite Guests →

- Prepare list
- Invite
- Confirm



Reactive Systems

A **reactive system** does NOT plan long-term.
It responds **immediately** to the environment.

Analogy

A mosquito swatter robot:

- If mosquito detected → swipe
- If no mosquito → idle

No big planning.

Examples

- Self-driving cars reacting to pedestrians
- Air conditioners adjusting temperature
- Robots avoiding obstacles

Section-3 (Expert Systems)

1. Expert Systems

Definition:

It is a computer program that emulate human experts. (OR) Software mimicking human experts (e.g., MYCIN for medical diagnosis).

- It Use specialized knowledge to solve problems
- It Provide explanations for reasoning

Think of an expert system as:

“A doctor inside a computer.”

You tell symptoms → it uses expert knowledge → gives diagnosis.

Example domains:

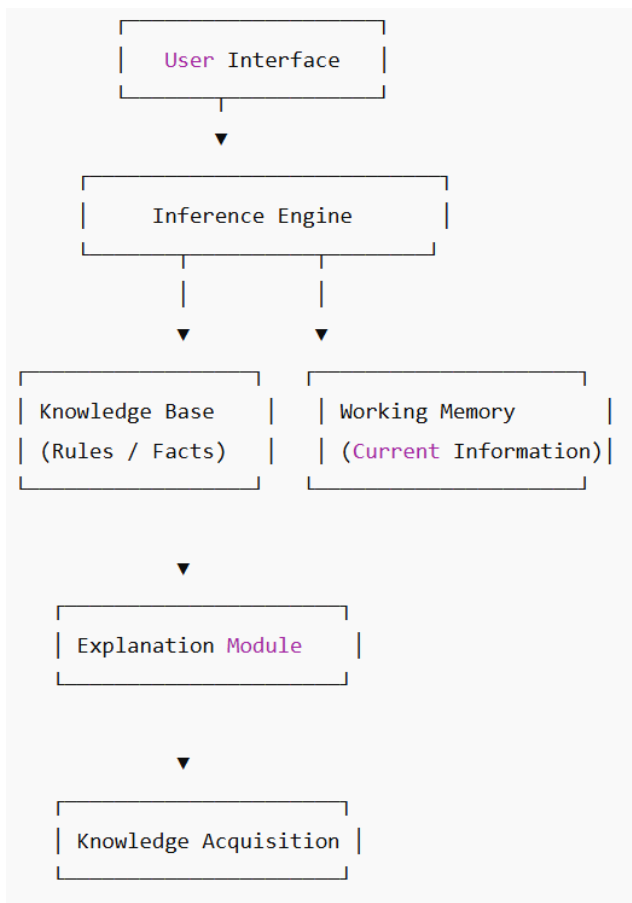
- Medical diagnosis
- Car engine troubleshooting
- Financial analysis
- Weather forecasting

2. Applications of Expert Systems

Real-world areas where expert systems are used:

Domain	Example Expert System	What it does
 Medicine	MYCIN	Diagnoses infections, suggests antibiotics
 Automobile	XCON	Helps configure automobiles / machines
 Finance	FINESSE	Loan approval, credit scoring
 Agriculture	CROP-HELP	Suggests fertilizers, pest solutions
 Mechanical	DART	Diagnoses faults in machinery
 Chemistry	DENDRAL	Identifies molecular structures
 Travel	Travel Expert	Suggests routes, plans trips

3. Components / Architecture of Expert Systems



Let us see the components one by one:

1. Knowledge Base

Contains **expert knowledge**:

- Facts ("Temperature = 102°F")
- Rules ("If fever + cough → flu")

2. Inference Engine (The Reasoning Mechanism)

The **brain** that:

- Matches rules
 - Applies logic
 - Draws conclusions
- Like a detective solving a case.

3. Working Memory

Working Memory = The place where the system stores the facts of the current situation.

It changes continuously while the expert system is reasoning.

Think of it as the "**notebook**" or **scratchpad** the expert system uses **RIGHT NOW** during problem solving.

Example: Imagine a **doctor** examining a patient.

- The doctor has many medical rules in their brain (Knowledge Base).

- But **for THIS patient**, the doctor writes these notes on a paper:
Patient has fever:
 - Temperature is 102°F
 - Patient has headache
 - Patient age is 15

That paper = **Working Memory**

The doctor's medical knowledge = **Knowledge Base**

The doctor's reasoning = **Inference Engine**

The doctor keeps updating the paper as symptoms are discovered.

WM usually contains entries like: (attribute, value)

Examples:

(symptom, fever)

(temperature, very high)

(blood_pressure, low)

(patient_age, 15)

4. User Interface

User interacts with system (questions, answers).

5. Explanation Module (Explains Reasoning)

Explains *why* the system gave a solution.

Example:

- "I diagnosed malaria because symptoms A, B, C matched rule R."

6. Knowledge Acquisition Module

How expert knowledge is entered:

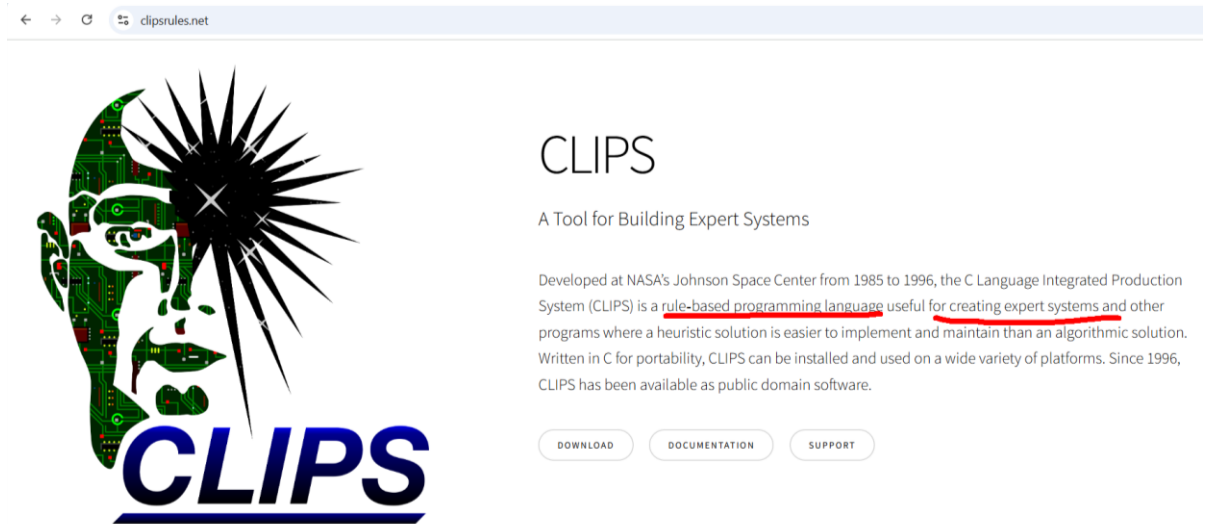
- Interviews with experts
- Machine learning results
- Data mining
- Add new knowledge

Expert System Shell: A tool containing the inference engine and user interface. A knowledge engineer adds the specific knowledge base to create a new expert system.

It is a difficult process of extracting knowledge from a human expert and coding it into the system.

Ex: **CLIPS** — a public-domain (free) expert system shell originally developed at NASA. It's widely used for building rule-based expert systems.

<https://www.clipsrules.net/>



4. Expert System Development

Steps in building an expert system:

Step 1: Problem Identification

Which domain?

Example: "Diagnose plant diseases."

Step 2: Knowledge Acquisition

Collect knowledge from:

- Domain experts
- Books
- Databases

Step 3: Knowledge Representation

Store knowledge as:

- Rules
- Frames
- Neural network weights

Step 4: Design Inference Engine

Forward chaining or backward chaining?

Example:

- Forward chaining: Data → Conclusion
- Backward chaining: Hypothesis → Check conditions

Step 5: Implementation

The following are the popular ways / platforms / IDEs:

CLIPS (C Language Integrated Production System) – Easy for Rule Based Systems

How CLIPS works

- You define **facts**
- You define **rules** using `defrule`
- CLIPS engine automatically does **forward chaining**

Note: Even though CLIPS is implemented in C Language, it follows LISP (List Processing) syntax.

Prolog (Logic based expert system)

- ✓ Best for backward-chaining systems
- ✓ Uses facts + rules expressed as logical clauses
- ✓ Natural fit for reasoning problems

How Prolog works

- You define facts
- You define rules (`:-` means IF)
- User asks a query
- Prolog tries to prove it using logic

Python + rule engine libraries

Python is now **very popular** for building expert systems because:

- ✓ Easy to write
- ✓ Easy to create GUIs
- ✓ Can integrate ML models (Hybrid Expert Systems)
- ✓ Good libraries available

“experta” is a popular Python tool for Rule-based engine (like CLIPS in C Language)

Step 6: Testing the Expert System

After implementation, the expert system must be **tested extensively**.

Testing ensures that:

- ✓ Rules fire correctly
- ✓ Inference engine works as intended
- ✓ Wrong conclusions are avoided
- ✓ The system behaves like a real human expert

There are different types of Testing, like:

Functional Testing

Does each rule fire correctly?

Example:

Input:

fever = yes

cough = yes

Output expected:

diagnosis = flu

Rule Conflict Testing

Ensure rules do NOT contradict.

Example:

Rule1:

IF cough THEN flu

Rule2:

IF cough THEN asthma

Testing reveals:

→ System may incorrectly output *two* diagnoses.

Fix:

Add more constraints, or ask user clarifying questions.

Boundary Testing

It is a software testing technique in which test cases are designed around the **boundaries** of input ranges rather than the middle values.

1. Expert systems often make life-critical decisions

- Medical diagnosis
- Drug dosage
- Industrial machine control
- Financial approval
- Fraud detection

2. Rules depend heavily on numeric thresholds

Small errors change the final conclusion.

3. Knowledge engineers may misunderstand domain limits

E.g., doctor says:

"Fever is above 100°F"

Engineer interprets:

"Rule: IF temp > 100"

But doctor meant:

"Temp ≥ 100"

Boundary testing catches this mismatch.

User Acceptance Testing (UAT)

Let **real domain experts** (doctors, lawyers, finance experts) interact with the system.

They report:

- ✓ Errors
- ✓ Misclassifications
- ✓ Missing knowledge
- ✓ Usability issues

Example feedback from a doctor:

“Your system says flu whenever there's fever + cough. But we also need duration and throat infection symptoms.”

This feedback → improves rule set.

Step 7: Refinement

Improve accuracy, update rules.

5. Program: Creating an Expert System using Python – Medical Diagnosis

Requirement: Creating a Medical Expert System that can diagnose:

- Flu
- Common Cold
- COVID-like symptoms
- Allergies

It contains **multiple rules, facts, conflict resolution**

Step1. Install required python package in Python 3.7

New versions like Python 3.10, 3.13 have bugs or no support.

```

C:\Users\bvraju\AppData\Local\Programs\Python\Python37\Scripts>pip.exe install experta
Collecting experta
  Downloading experta-1.9.4-py3-none-any.whl (35 kB)
Collecting frozendict==1.2
  Downloading frozendict-1.2.tar.gz (2.6 kB)
Collecting schema==0.6.7
  Downloading schema-0.6.7-py2.py3-none-any.whl (14 kB)
Building wheels for collected packages: frozendict
  Building wheel for frozendict (setup.py) ... done
  Created wheel for frozendict: filename=frozendict-1.2-py3-none-any.whl size=3146 sha256=d00aefa163e41fc60f5569c5f0ea
a598d43842db16a0b4a33fbfd180d9178ac1
  Stored in directory: c:\users\bvraju\AppData\Local\pip\cache\wheels\68\17\69\ac196dd181e620bba5fae5488e4fd6366a7316d
ce13cf88776
Successfully built frozendict
Installing collected packages: schema, frozendict, experta
Successfully installed experta-1.9.4 frozendict-1.2 schema-0.6.7
WARNING: You are using pip version 21.0.1; however, version 24.0 is available.
You should consider upgrading via the 'c:\users\bvraju\AppData\Local\Programs\Python\Python37\python.exe -m pip instal
l --upgrade pip' command.

C:\Users\bvraju\AppData\Local\Programs\Python\Python37\Scripts>

```

Step2. Write the following program

```

from experta import *

# -----
# Knowledge Engine for Diagnosing Common Illnesses
# -----

class MedicalDiagnosis(KnowledgeEngine):

    # RULE 1: If fever + cough + body pain → Flu
    @Rule(Fact(fever=True),
          Fact(cough=True),
          Fact(body_pain=True),
          salience=5)
    def diagnose_flu(self):
        print("Rule Fired → Diagnosing Flu")
        self.declare(Fact(diagnosis="Flu"))

    # RULE 2: If runny nose + sneezing + no fever → Common Cold
    @Rule(Fact(runny_nose=True),
          Fact(sneezing=True),
          NOT(Fact(fever=True)),
          salience=5)
    def diagnose_cold(self):
        print("Rule Fired → Diagnosing Common Cold")
        self.declare(Fact(diagnosis="Common Cold"))

    # RULE 3: If fever + cough + loss of smell → COVID-like symptoms
    @Rule(Fact(fever=True),
          Fact(cough=True),
          Fact(loss_of_smell=True),
          salience=10)
    def diagnose_covid(self):
        print("Rule Fired → Diagnosing COVID-like Symptoms")
        self.declare(Fact(diagnosis="COVID-like symptoms"))

```

```

# RULE 4: If itchy eyes + sneezing + runny nose → Allergies
@Rule(Fact(itchy_eyes=True),
      Fact(sneezing=True),
      Fact(runny_nose=True),
      salience=3)
def diagnose_allergy(self):
    print("Rule Fired → Diagnosing Allergy")
    self.declare(Fact(diagnosis="Allergy"))

# RULE 5: If sore throat + cough → Throat infection
@Rule(Fact(sore_throat=True),
      Fact(cough=True),
      salience=2)
def diagnose_throat(self):
    print("Rule Fired → Diagnosing Throat Infection")
    self.declare(Fact(diagnosis="Throat Infection"))

# RULE 6: If no rule fired → Unknown illness
@Rule(NOT(Fact(diagnosis=MATCH.d)),
      salience=1)
def diagnose_unknown(self):
    print("Rule Fired → Diagnosing as Unknown")
    self.declare(Fact(diagnosis="Unknown - need more symptoms"))

# -----
# Running the Expert System
# -----

engine = MedicalDiagnosis()
engine.reset() #clears all previous facts so you start clean.
#Now engine is ready to accept new facts (New Patient's data)

# Declare a patient's symptoms
engine.declare(Fact(fever=True))
engine.declare(Fact(cough=True))
engine.declare(Fact(loss_of_smell=False))
engine.declare(Fact(body_pain=True))
engine.declare(Fact(runny_nose=False))
engine.declare(Fact(sneezing=False))
engine.declare(Fact(itchy_eyes=False))
engine.declare(Fact(sore_throat=False))

engine.run()

```

Step3. Run the Program

Output:

```

Rule Fired → Diagnosing Flu
>>> |

```

Change the Facts:

```

# -----
# Running the Expert System
# -----

engine = MedicalDiagnosis()
engine.reset() #clears all previous facts so you start clean.
#Now engine is ready to accept new facts (New Patient's data)

# Declare a patient's symptoms
engine.declare(Fact(fever=False))
engine.declare(Fact(cough=True))
engine.declare(Fact(loss_of_smell=False))
engine.declare(Fact(body_pain=True))
engine.declare(Fact(runny_nose=False))
engine.declare(Fact(sneezing=True))
engine.declare(Fact(itchy_eyes=False))
engine.declare(Fact(sore_throat=True))

engine.run()

```

New Output:

```

Rule Fired → Diagnosing Throat Infection
>>> |

```

6. Program: Creating an Expert System using Prolog – Medical Diagnosis

Step1. Open <https://swish.swi-prolog.org/>

Step2. Create a new file and write the following code

```

1 % -----
2 % Medical Diagnosis Expert System in Prolog
3 % -----
4
5 % RULE 1: Fever + Cough + Body Pain → Flu
6 diagnosis("Flu") :-
7     fever(true),
8     cough(true),
9     body_pain(true),
10    !. % Cut: highest priority (similar to salience)
11
12 % RULE 2: Runny nose + Sneezing + No fever → Common Cold
13 diagnosis("Common Cold") :-
14     runny_nose(true),
15     sneezing(true),
16     fever(false),
17     !.
18
19 % RULE 3: Fever + Cough + Loss of Smell → COVID-like
20 diagnosis("COVID-like symptoms") :-
21     fever(true),
22     cough(true),
23     loss_of_smell(true),
24     !.
25

```



```

26 % RULE 4: Itchy eyes + Sneezing + Runny nose → Allergy
27 diagnosis("Allergy") :-
28     itchy_eyes(true),
29     sneezing(true),
30     runny_nose(true),
31     !.
32
33 % RULE 5: Sore throat + Cough → Throat Infection
34 diagnosis("Throat Infection") :-
35     sore_throat(true),
36     cough(true),
37     !.
38
39 % RULE 6: If nothing matches → Unknown illness
40 diagnosis("Unknown - need more symptoms").
41
42 fever(false).
43 cough(true).
44 loss_of_smell(false).
45 body_pain(true).
46 runny_nose(false).
47 sneezing(true).
48 itchy_eyes(false).
49 sore_throat(true).
50

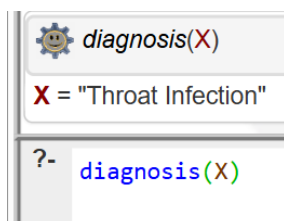
```

Note:

In Prolog, the symbol **!** is called a **cut**.
It is *not* a priority value like salience in Experta/CLIPS.

It is a **control operator** that tells Prolog:
“Stop looking for any other rules once this rule succeeds.”

Step3. Execute the following query



7. Types of Expert Systems

- **Rule-Based Expert Systems:** Expertise is stored as *IF–THEN rules*, and reasoning happens through rule matching.
- **Frame-Based Systems:** Knowledge is stored as **objects/frames** with properties and inheritance.
- **Fuzzy Expert Systems:** They use **degrees of truth** instead of strict true/false logic.
- **Neural Network Based Expert Systems:** The system learns expert knowledge from **data** using neural networks.
- **Hybrid Expert Systems:** Combines 2 or more AI techniques (rules + fuzzy + neural networks + frames).

- **Model-Based Expert Systems:** It uses a blueprint or internal model of how a system works. It reasons logically based on structure, not rules.
- **Case-Based Reasoning Systems:** The system solves new problems by **looking at old similar problems** in its memory. It works like a **doctor remembering old patients**, so that the same treatment can be used for new patients.

8. Rule-Based Expert Systems (Most common)

Uses **IF–THEN** rules.

Ex:

```
IF fever AND body pain AND chills
THEN diagnosis = "Viral Infection".
```

How it works:

```
Facts → Match with rules → Fire rule → Conclusion
```

Example:

Facts:

- Fever = 102°F
- Body pain = Yes
- Chills = Yes

Rule fires:

→ Viral infection.

Forward or backward chaining

Examples: MYCIN (medical diagnosis), XCON (computer config), DENDRAL (chemical analysis).

9. Frame-Based Expert System

Frames = **structured knowledge objects** like objects in Object Oriented Program.

Example: A “Car” Frame

```
Frame: Car
Slots:
  Wheels: 4
  Engine: Petrol
  Transmission: Automatic
```

Frame for a “SportsCar” may **inherit** slots:

```
Frame: SportsCar
  IS-A: Car
  Slots:
    Top-Speed: 300 km/h
```

Frames = “Templates / Blueprints”

Slots = “Properties / Attributes”

10. Neural Network Based Expert System

These systems use:

- **Neural networks to learn from examples**
- **Expert knowledge embedded as weights**

Example:

- A neural net trained to classify skin diseases.

Advantage:

Learns from data automatically.

Disadvantage:

Cannot **explain reasoning**

These are the main properties:

- Learn from examples
- Pattern recognition – edges, skin textures, etc
- No explicit rules

Rule based Vs Model Based Expert Systems

Rule-Based Expert System

Programmer must write explicit rules:

```
IF no-electricity THEN fan-won't-work
IF switch-off THEN fan-won't-work
IF motor-damaged THEN fan-won't-work
IF regulator-fault THEN fan-won't-work
```

If a problem occurs that has **no rule**,
the system **cannot diagnose it**.

Model-Based Expert System

The system knows the fan's structure:

Fan works **when**:
electricity **is** OK
AND **switch is** ON
AND motor **is** OK
AND regulator **is** OK

Now if the fan stops:

- It checks each component logically.
- It can detect new faults **even if no rule exists**.

Example:

Loose wire → not in rules.

Model-based system still finds:

“Electricity not reaching motor → fault in wiring.”

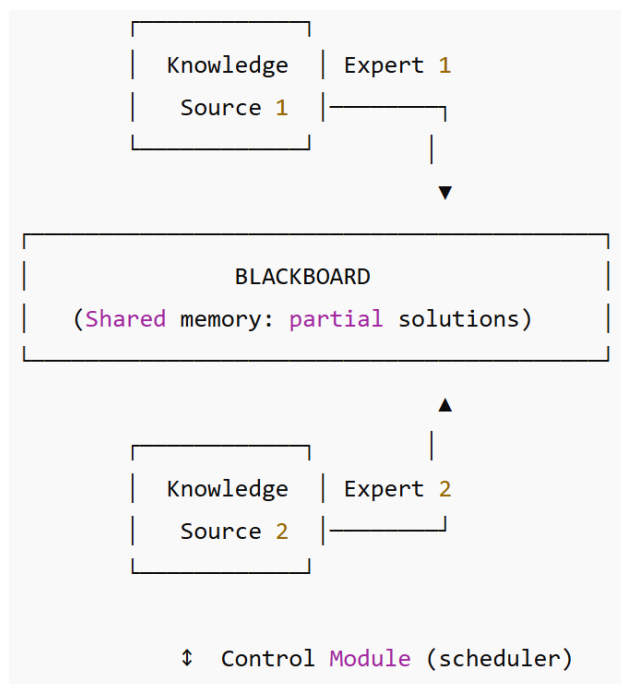
11. Blackboard Architecture (Multiple knowledge sources)

A powerful architecture used when:

- Problem is complex
- Many experts/sensors contribute partial solutions

Imagine a classroom:

- A large blackboard (shared memory)
- Many experts write suggestions
- A controller manages who writes next



Example:

Speech recognition system:

- One expert processes phonemes
- Another expert uses grammar
- Another expert matches words

Their combined work improves accuracy.

Blackboard architecture is a cooperative multi-expert system where a central scheduler decides which expert module should act next based on changes in a shared blackboard.

Why do we need a Scheduler?

Because multiple experts:

- do not know each other
- cannot coordinate directly
- may create conflict
- may produce redundant work
- may need different priorities

The **scheduler keeps them in order**, prevents chaos, and ensures productive cooperation.

Example: Self-Driving Car Perception System

Self-driving cars (Tesla, Waymo, Mercedes, Volvo) use a **blackboard architecture** to interpret the environment and make decisions.



Why?

Because a car receives **massive, continuous, conflicting, and incomplete data** from many sensors:

- Cameras
- LIDAR
- Radar
- GPS
- Ultrasonic sensors
- IMU (Inertial Measurement Unit)

These sensors must work *together* to produce:

- Lane detection
- Vehicle recognition
- Pedestrian detection
- Traffic sign interpretation

- Speed estimation
- Collision prediction

A **blackboard architecture** allows all these expert systems to cooperate.

12. Case Studies of Successful Expert Systems

1. MYCIN (Medical diagnosis)

- A Rule based expert system developed at Stanford in the 1970s
- Diagnosed bacterial infections
- More accurate than junior doctors
- It also recommended **antibiotics + dosage** accurately.

2. DENDRAL (Chemistry)

- It used **domain knowledge** about chemistry – like chemical bonds, possible molecular arrangements
- Identified chemical structures using spectral data
- Used by pharmaceutical researchers to speed up drug discovery.

3. XCON (Computer configuration)

- Used by DEC (Digital Equipment Corporation) in 1980s for assembling VAX computers
- Reduced configuration mistakes.
- Saved millions of dollars annually
- It used **thousands of rules** about hardware compatibility.

4. PROSPECTOR (Geology)

- Developed in the 1970s, PROSPECTOR analyzed geological data to identify likely mineral-rich sites.
- PROSPECTOR guided a mining company to discover a **major molybdenum metal (useful for making high-strength steel) deposit in Washington**, worth millions.
- Guided mining companies
- It proved expert systems can make real-world discoveries

13. Knowledge Acquisition and Validation Techniques

Knowledge Acquisition

Collecting expert knowledge.

Methods:

1. **Interviews** with domain experts
2. **Observation** of expert's work
3. **Protocol analysis** (expert explains while working)
4. **Questionnaires**
5. **Data mining** (automatic)
6. **Machine learning models**

Knowledge Validation

Ensuring correctness.

Methods:

- Test with real cases
- Ask experts to review rules
- Debug rules
- Consistency checks (no contradictions)
- Completeness checks (no missing rules)